CLOUD MUSIC SUBSCRIPTION APP

AWS Cloud System Development Report

Abstract

This report presents our group's AWS-based Cloud Music Subscription App, including the React frontend, DynamoDB and S3 data layer, and three backend deployments using EC2, ECS/Fargate, and API Gateway with Lambda.

Group 121

Aditya Rajbhoj | s4097885

Anirudha Bhoir | s4143390

Chaitanya Parab | s4068013

Kanika Sharma | s4108453

Table of Contents

1. Introduction ***2***

2. System Architecture Overview ***2***

2.1. Overall Architecture 2
2.2. Application Data Flow 3
2.3. Deployment Summary 4

3. Data Storage Architecture: DynamoDB and S3 ..... ***6***

3.1. Dataset Analysis and Storage Requirements 6
3.2. DynamoDB Table Design 7
3.3. Indexing Strategy: GSI, LSI, Query, and Scan 8
3.4. S3 Artist Image Storage 9
3.5. Data Layer Validation 10

4. Frontend Architecture and S3 Static Hosting ***10***

4.1. Frontend Design 10
4.2. Backend Endpoint Configuration 11
4.3. S3 Static Website Hosting 12
4.4. Frontend Functional Validation 13

5. Backend Deployment Architectures ***14***

5.1. EC2 Deployment 14
5.2. ECS/Fargate Deployment 16
5.3. API Gateway and Lambda Deployment 17

6. Functional Testing and Validation ***18***

7. Critical Comparison of Backend Architectures ***20***

8. Security, Cost, Scalability, and Operational Considerations ***22***

8.1. Security Considerations 22
8.2. Cost Considerations 23
8.3. Scalability Considerations 23
8.4. Operational Considerations 24

9. Limitations and Future Improvements ***25***

10. Conclusion ***25***

11. References ***26***

1. Introduction

This report presents the design, deployment, and validation of the **Cloud Music Subscription App**, an AWS-based online music subscription web application developed for Cloud Computing Assessment 2. The application allows users to register, log in, search music records, subscribe to songs, remove subscriptions, and view artist images stored in Amazon S3.

The system integrates **Amazon DynamoDB** for login, music, and subscription data, **Amazon S3** for artist image storage and static frontend hosting, and three independently deployed backend architectures: **Amazon EC2**, **Amazon ECS/Fargate**, and **API Gateway with AWS Lambda**. The frontend was implemented using React and configured to connect to each backend deployment for testing.

A key design focus was preserving the integrity of the `2026a2_songs.json` dataset. Since the dataset contains duplicate song titles and overlapping metadata, the DynamoDB music table was designed with a composite key structure, a generated `song_id`, and purposeful GSI/LSI indexes to support efficient search without accidental overwrites.

The completed implementation was validated through health checks, login, registration, music search, subscription creation, subscription retrieval, and subscription deletion. This report explains the architecture, justifies the database and deployment decisions, presents testing evidence, and compares the three backend approaches in terms of scalability, cost, operational complexity, performance, and maintainability.

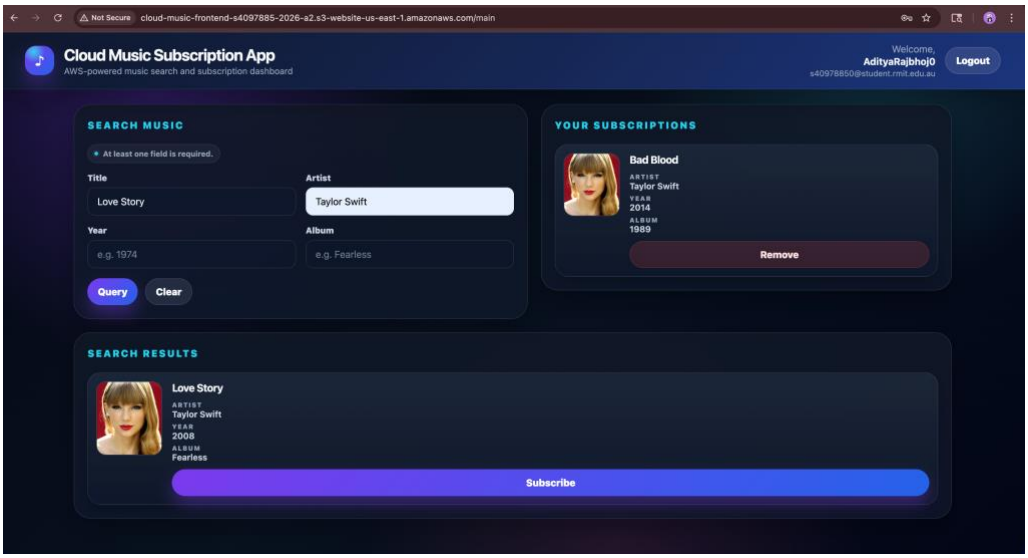

Figure 1. Cloud Music Subscription App frontend interface showing the AWS-backed dashboard, music search function, subscription area, and artist image display.

2. System Architecture Overview

2.1. Overall Architecture

Our system was designed as a modular AWS-based music subscription application with separate frontend, backend, database, and storage layers. The React frontend is hosted as a

static website on Amazon S3 and communicates with the backend through HTTP API requests. The backend is deployed in three independent ways: Amazon EC2, Amazon ECS/Fargate, and API Gateway with AWS Lambda. Each backend implementation supports the same application functions and connects to the same DynamoDB tables and S3 artist image bucket.

This structure allowed us to keep the application behaviour consistent while testing different backend deployment models. The frontend can be connected to any backend deployment by rebuilding it with the required `VITE_API_BASE_URL`. During final validation, we used the API Gateway endpoint as the demo-ready backend because it demonstrates the serverless implementation while still using the same DynamoDB and S3 resources as the EC2 and ECS versions.

The data layer uses three DynamoDB tables: `login`, `music`, and `subscriptions`. The `login` table stores user credentials and usernames, the `music` table stores the imported song dataset, and the `subscriptions` table stores the songs selected by each user. Amazon S3 is used for two purposes: hosting the compiled React frontend and storing downloaded artist images. The backend returns S3 image URLs with music records so that the frontend can display artist images in the search results and subscription area.

This architecture separates presentation, application logic, data storage, and object storage while allowing a direct comparison of virtual server, containerised, and serverless backend models.

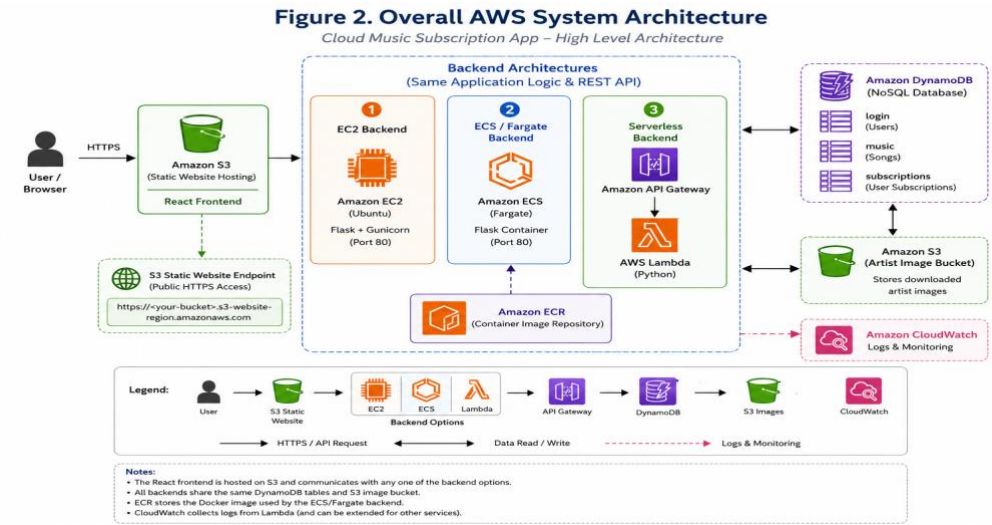

Figure 2. Overall AWS system architecture showing the S3-hosted React frontend, EC2 backend, ECS/Fargate backend, API Gateway and Lambda backend, DynamoDB tables, S3 artist image bucket, ECR repository, and CloudWatch logging.

2.2.Application Data Flow

The application flow begins when a user accesses the React frontend from the S3 static website URL. After login, the frontend sends requests to the configured backend endpoint for registration, music search, subscription creation, subscription retrieval, subscription deletion, and logout. The backend then performs the required DynamoDB operation and returns a structured JSON response to the frontend.

For authentication, the backend checks the submitted email and password against the `login` table. For registration, it first checks whether the email already exists before inserting a new user. For music search, the backend accepts title, artist, year, and album fields, treating multiple fields as AND conditions. The search logic uses DynamoDB Query operations where suitable, including the `title-index` GSI and `year-album-index` LSI, while controlled Scan is reserved for broader retrieval patterns.

For subscriptions, the backend writes or deletes records in the `subscriptions` table using the user's email and the selected `song_id`. When a user subscribes to a song, the record is stored in DynamoDB and appears in the frontend subscription area. When the user removes a song, the corresponding DynamoDB record is deleted and the frontend refreshes the subscription list. This confirms that the application performs the required create, read, and delete operations through the deployed backend APIs.

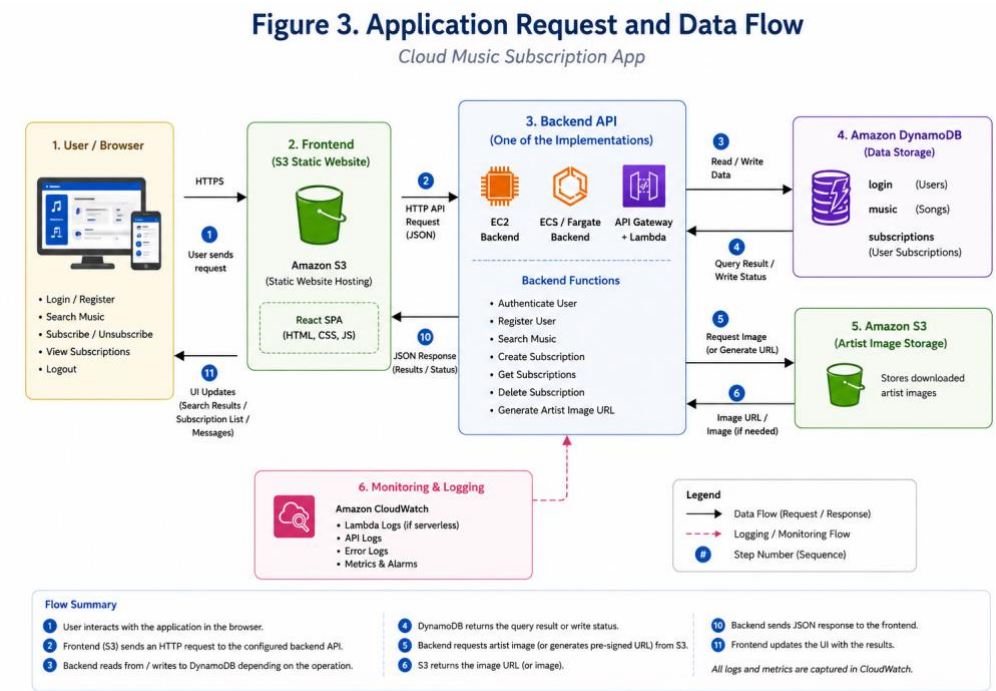

Figure 3. Application request and data flow showing how the user interacts with the S3-hosted React frontend, how requests are sent to the configured backend API, how DynamoDB is used for login, music, and subscription data, and how S3 artist image URLs are returned for frontend display.

2.3.Deployment Summary

Our implementation includes four main deployed components. The frontend is served from Amazon S3 as static React assets. The EC2 backend runs the Flask application with Gunicorn on standard HTTP port 80. The ECS backend packages the same application into a Docker image, stores it in Amazon ECR, and deploys it as a Fargate service. The serverless backend uses API Gateway routes mapped to a native Lambda function.

All three backend deployments were tested independently to confirm equivalent behaviour. This was important because the application needed to demonstrate the same core functions

across different cloud deployment models rather than relying on a single backend implementation.

Table 1. Summary of deployed AWS components

Layer	AWS service	Purpose in our implementation
Frontend	Amazon S3 static website hosting	Hosts the compiled React application
Backend 1	Amazon EC2	Runs Flask/Gunicorn backend on a virtual server
Backend 2	Amazon ECS/Fargate and Amazon ECR	Runs the backend as a containerised service
Backend 3	API Gateway and AWS Lambda	Provides the serverless REST-style backend API
Database	Amazon DynamoDB	Stores login, music, and subscription records
Object storage	Amazon S3	Stores frontend assets and artist images
Monitoring	Amazon CloudWatch	Records Lambda execution logs and supports troubleshooting

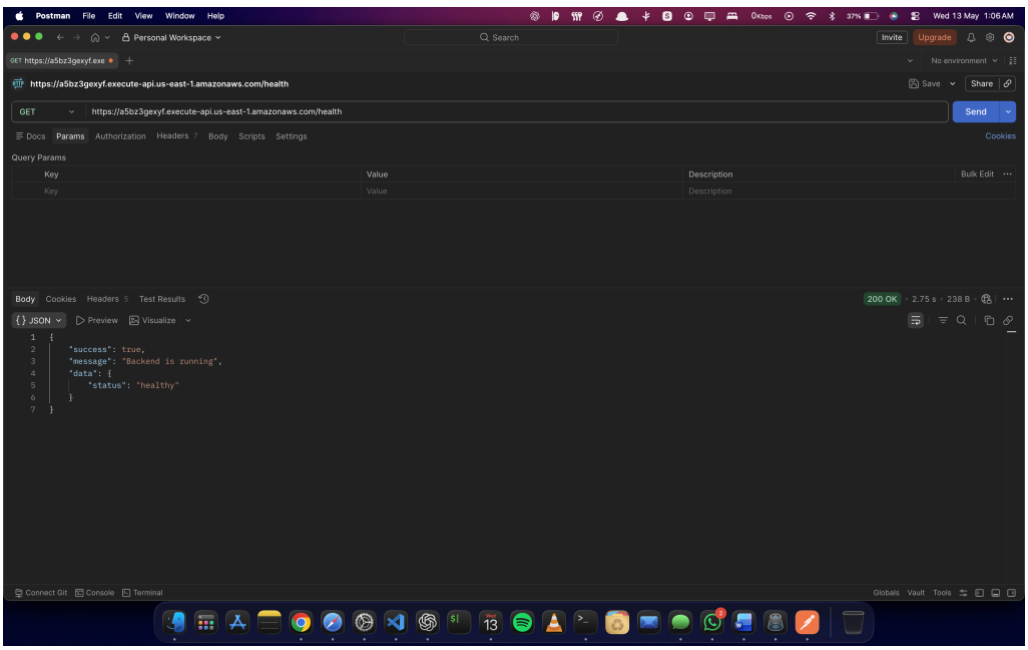

Figure 4. API Gateway health-check output confirming that the serverless backend is reachable and successfully invoking the Lambda function.

3. Data Storage Architecture: DynamoDB and S3

3.1. Dataset Analysis and Storage Requirements

Our data layer was designed after analysing the structure of the `2026a2_songs.json` dataset. The dataset contains music records with the fields `title`, `artist`, `year`, `album`, and `img_url`. During analysis, we identified that the dataset contains repeated song titles and overlapping artist, album, and year combinations. This meant that using `title` alone as a primary key would be unsafe because later records with the same title could overwrite earlier records.

To prevent this issue, our implementation generated a stable `song_id` from the combination of title, artist, year, and album. This gave each music record a unique identifier while still preserving the original dataset fields. The final imported dataset contained 137 music records, and the database verification script confirmed that all records were stored successfully. This design choice was important because the application needed to support accurate searching, subscribing, and deletion without losing records due to duplicate titles.

The database layer was separated into three DynamoDB tables: `login`, `music`, and `subscriptions`. This separation made the design easier to manage because user authentication, music retrieval, and subscription records each have different access patterns. It also allowed us to test each table independently during validation.

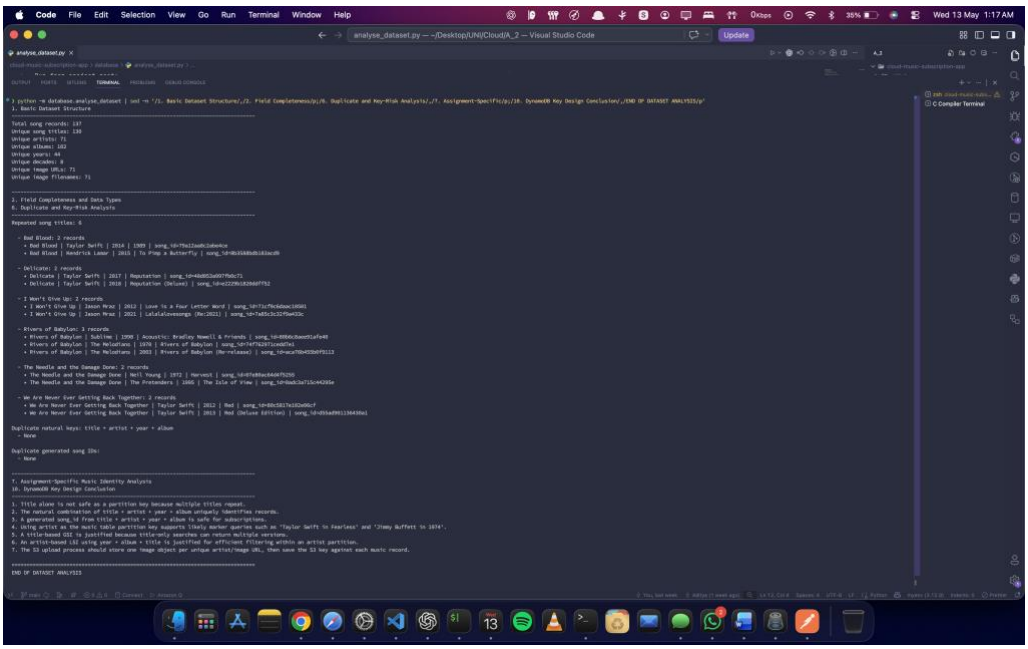

Figure 5. Dataset analysis output showing the dataset size, duplicate title risks, generated song ID uniqueness, and DynamoDB key design conclusion

Table 2. Dataset analysis summary

Dataset characteristic	Result	Design implication
Total song records	137	All records needed to be preserved during import
Unique song titles	130	Some titles repeat and cannot be used as a unique key
Unique artists	71	Artist-based partitioning supports common search patterns
Unique albums	102	Album needed to remain part of song identity
Unique image URLs	71	One artist image object could be stored per unique image URL
Repeated song titles	6	Title-only keys risk overwriting or ambiguous retrieval
Duplicate generated song IDs	0	Generated `song_id` was safe for subscription records

3.2.DynamoDB Table Design

The `login` table stores user account records using `email` as the partition key. This design allows the backend to retrieve a user directly during login or registration checks. When a user attempts to register, the backend first checks whether the email already exists. If it does not exist, a new user record is written to the table.

The `music` table stores the imported song records. Our implementation uses `artist` as the partition key and a composite sort key based on song metadata. This supports efficient artist-based searches and avoids storing records in a way that would collapse duplicate titles. Each item also includes a generated `song_id`, which is used by the subscription system to identify the exact song selected by a user.

The `subscriptions` table stores the relationship between users and songs. It uses `email` as the partition key and `song_id` as the sort key. This allows the backend to retrieve all subscriptions for a user and delete a specific subscription without scanning the whole table. This table design directly supports the subscription area in the frontend, where each logged-in user sees only their own subscribed songs.

Table 3. DynamoDB table design

Table	Partition key	Sort key	Purpose
`login`	`email`	None	Stores registered users and supports login/registration checks
`music`	`artist`	Composite song metadata key	Stores imported music records without overwriting duplicate titles

Table	Partition key	Sort key	Purpose
`subscriptions`	`email`	`song_id`	Stores each user's subscribed songs

3.3.Indexing Strategy: GSI, LSI, Query, and Scan

The `music` table was designed with a purposeful indexing strategy to support the search requirements of the application. Since users can search by title, artist, year, and album, the backend needs to handle different query combinations. Our implementation uses a combination of DynamoDB Query operations, a Global Secondary Index, a Local Secondary Index, and controlled Scan operations.

The `title-index` Global Secondary Index supports title-based searches. This is necessary because `title` is not the main table partition key, but title search is one of the required query options in the frontend. The GSI allows the backend to retrieve songs by title without relying only on a full table scan.

The `year-album-index` Local Secondary Index supports search patterns within an artist partition, such as retrieving an artist's songs by year or album. This is useful for queries such as `Taylor Swift` with album `Fearless` or `Jimmy Buffett` with year `1974`. These examples were used during database verification and API testing to confirm that indexed retrieval was working correctly.

Controlled Scan is still included for broader search cases where the input combination does not match a direct index pattern. However, our implementation does not rely on Scan as the default for every request. Instead, it uses Query where the key pattern supports it and Scan only when needed. This makes the retrieval logic more efficient and better aligned with DynamoDB design principles.

Table 4. Music table indexing strategy

Access pattern	Retrieval approach	Reason
Search by artist	Query on main table partition key	Efficient direct retrieval using `artist`
Search by title	Query using `title-index` GSI	Supports title lookup even though title is not the main partition key
Search by artist and year/album	Query using LSI-supported sort pattern	Supports refined searches within an artist partition

Access pattern	Retrieval approach	Reason
Broader mixed-field search	Controlled Scan with filters	Used only when the query pattern is not fully supported by keys

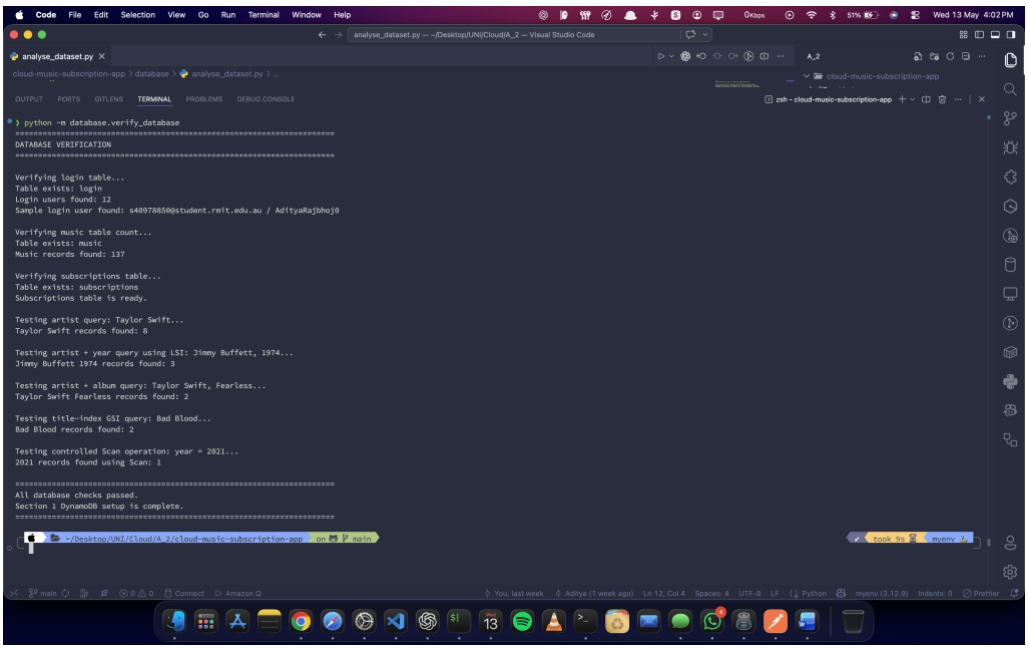

Figure 6. DynamoDB verification output showing successful validation of the login, music, and subscriptions tables, including table counts, GSI title lookup, LSI artist-year lookup, controlled Scan operation, and overall database readiness.

3.4.S3 Artist Image Storage

Amazon S3 was used to store artist images separately from DynamoDB. The original dataset included external image URLs, but relying on external URLs would make the application dependent on third-party availability. To improve control and consistency, our implementation downloaded the artist images and uploaded them to an S3 bucket under the `artist-images/` prefix.

After the images were uploaded, each music record in DynamoDB was updated with an `s3_image_key`. This allowed the backend to generate an image URL whenever a music record was returned to the frontend. As a result, the frontend can display artist images in both the search results and subscription area without needing to store image files directly in the frontend application.

This approach also keeps the database lightweight. DynamoDB stores only metadata and image keys, while S3 stores the actual image objects. This is a better design than storing images in the database because image files are object data and are more suitable for S3.

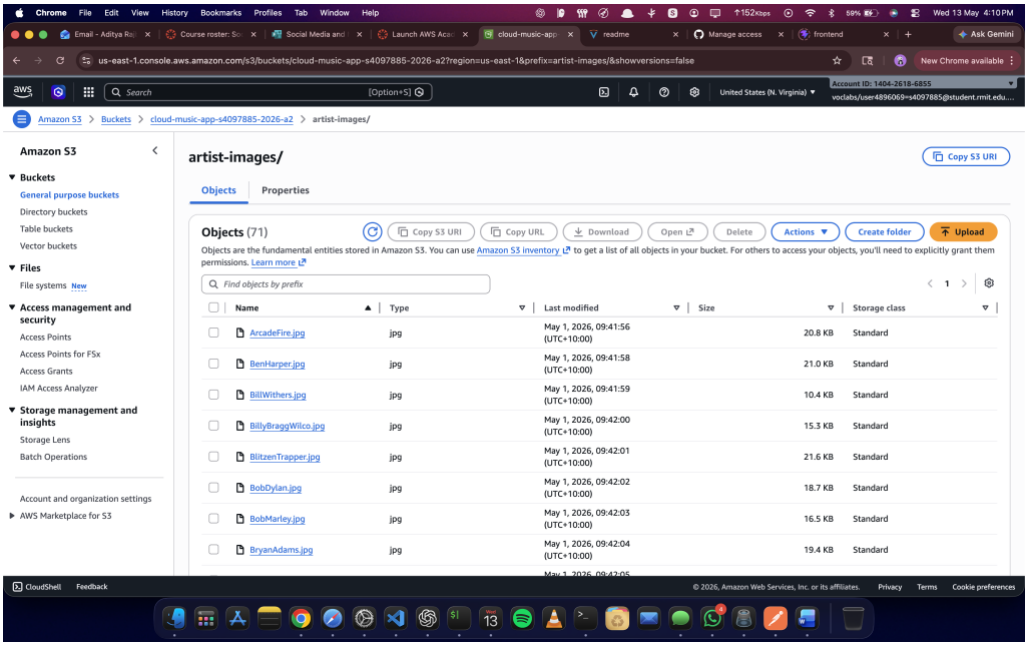

Figure 7. Amazon S3 artist image bucket showing 71 uploaded artist image objects stored under the `artist-images/` prefix for backend image URL generation.

3.5.Data Layer Validation

We validated the data layer through both database scripts and API-level testing. The `verify_database.py` script confirmed that the `login`, `music`, and `subscriptions` tables were available and that expected query patterns returned valid results. It also confirmed that all 137 music records were present after import.

The API tests further confirmed that the deployed backends could read from and write to DynamoDB. Login tests confirmed access to the `login` table, music search tests confirmed access to the `music` table and indexes, and subscription tests confirmed create, read, and delete operations on the `subscriptions` table. The successful frontend display of artist images also confirmed that S3 image keys and URL generation were working correctly.

Overall, the data layer design supported both correctness and usability. DynamoDB provided structured storage for users, songs, and subscriptions, while S3 provided scalable object storage for artist images. Together, these services formed a reliable foundation for the frontend and all three backend deployment models.

4. Frontend Architecture and S3 Static Hosting

4.1. Frontend Design

Our frontend was implemented as a React single-page application to provide a simple and responsive interface for the music subscription system. The frontend includes pages and components for user login, new user registration, music search, subscription management, and logout. This design allowed us to keep the user experience consistent while changing the backend endpoint during deployment testing.

The frontend is organised around reusable components rather than one large page. Components such as the header, query form, results panel, song card, subscription panel, and message banner separate the interface into clear functional areas. This made the interface easier to maintain and allowed the same song display structure to be reused for both search results and subscribed songs.

The main dashboard contains three required user-facing areas. The user area displays the logged-in user and provides logout functionality. The query area allows users to search by title, artist, year, and album. The subscription area displays the songs currently subscribed by the logged-in user. Search results and subscriptions are updated dynamically after API calls, so the user can immediately see the result of actions such as subscribing to or removing a song.

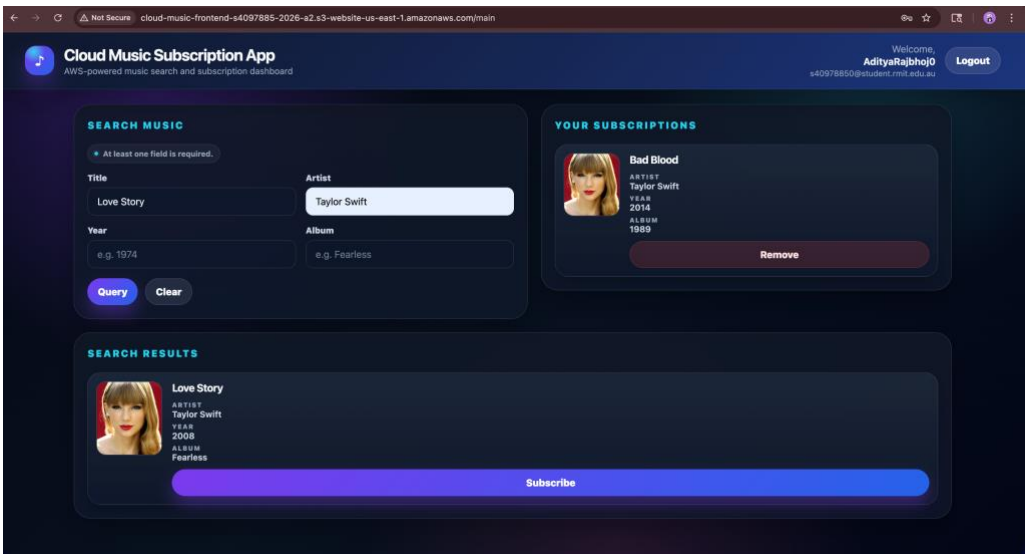

Figure 8. Cloud Music Subscription App dashboard showing the logged-in user area, search form, search results, subscription panel, and artist images returned through the AWS-backed application.

Figure 8 shows the completed frontend interface after login. The dashboard provides the required application areas in one page, allowing the user to search the music catalogue, view returned songs with artist images, manage subscriptions, and log out.

4.2.Backend Endpoint Configuration

The frontend was designed to connect to different backend deployments using the `VITE_API_BASE_URL` environment variable. This was important because the project required the backend to be deployed using EC2, ECS/Fargate, and API Gateway with Lambda. Instead of changing frontend source code for each backend, we configured the backend URL at build time and redeployed the static frontend assets to S3.

For local testing, the frontend can connect to the Flask backend running on `http://127.0.0.1:5050`. For cloud testing, it can connect to the EC2 public IP, ECS/Fargate public IP, or API Gateway endpoint. During final validation, we deployed the frontend using the API Gateway URL because it provided the clearest fully managed cloud flow from S3 frontend to API Gateway, Lambda, DynamoDB, and S3 image storage.

This configuration approach made the frontend deployment flexible. It also supported independent testing of each backend architecture while keeping the same user interface and application behaviour.

Table 5. Frontend backend configuration options

Deployment target	Example backend URL format	Purpose
Local Flask backend	`http://127.0.0.1:5050`	Local development and functional testing
EC2 backend	`http://EC2_PUBLIC_IP`	Tests the virtual-server backend deployment
ECS/Fargate backend	`http://ECS_PUBLIC_IP`	Tests the containerised backend deployment
API Gateway + Lambda backend	`https://API_GATEWAY_URL`	Tests the serverless backend deployment

4.3.S3 Static Website Hosting

The final frontend build was deployed to an Amazon S3 bucket configured for static website hosting. The React application was built using Vite, which generated static HTML, CSS, JavaScript, and asset files in the `dist.` directory. These files were then uploaded to the S3 frontend bucket using the deployment script.

S3 static hosting was suitable for this application because the frontend does not require server-side rendering. All dynamic behaviour is handled through API calls to the configured backend. This keeps the frontend hosting simple, low-cost, and easy to redeploy whenever the backend endpoint changes.

The deployment script performs the required frontend build steps, creates or reuses the S3 bucket, enables static website hosting, applies the bucket policy needed for public website access, and uploads the built frontend files. This reduced manual deployment errors and made the frontend deployment repeatable for demonstration and testing.

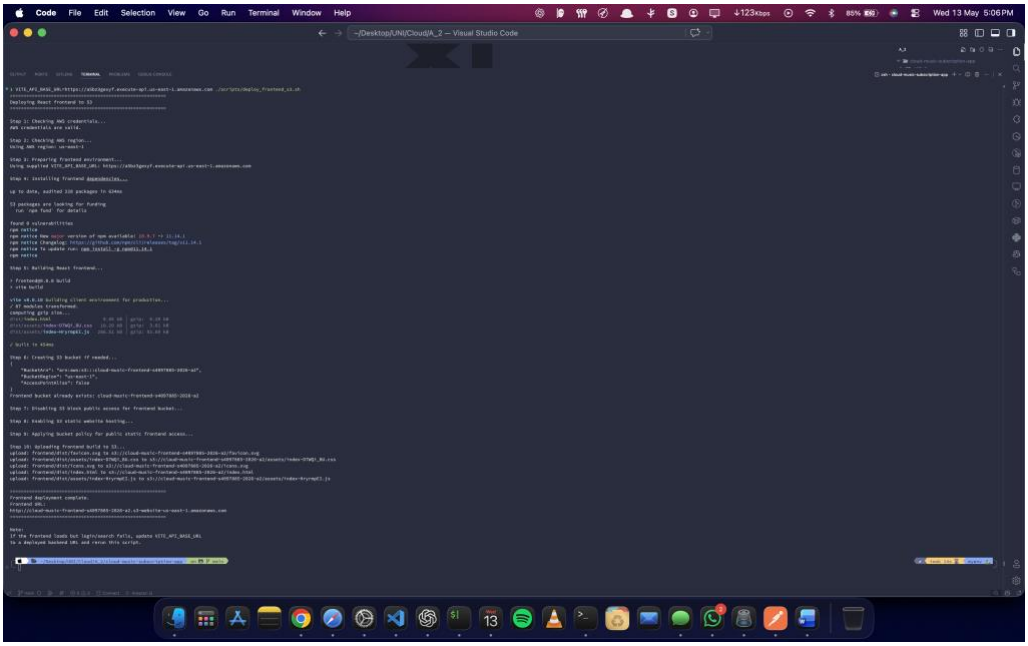

Figure 9. Frontend S3 deployment output showing the React production build uploaded to the S3 static website bucket and the final frontend website URL.

Figure 9 provides deployment evidence for the frontend layer. The output confirms that the React application was built successfully and uploaded to the S3 bucket used for static website hosting.

4.4.Frontend Functional Validation

The frontend was validated by testing the main user workflows through the deployed S3 website. These workflows included login, registration, music search, subscription creation, subscription removal, and logout. The browser-based tests confirmed that the frontend could send requests to the configured backend endpoint and update the interface using the returned JSON responses.

The successful display of artist images also confirmed that the frontend received image URLs generated by the backend and loaded the corresponding image objects from S3. This was an important validation point because the application depends on coordination between the frontend, backend API, DynamoDB metadata, and S3 image storage.

For final validation, the S3 frontend was connected to the API Gateway endpoint. This confirmed that the full serverless path was working from the user interface through API Gateway and Lambda to DynamoDB and S3.

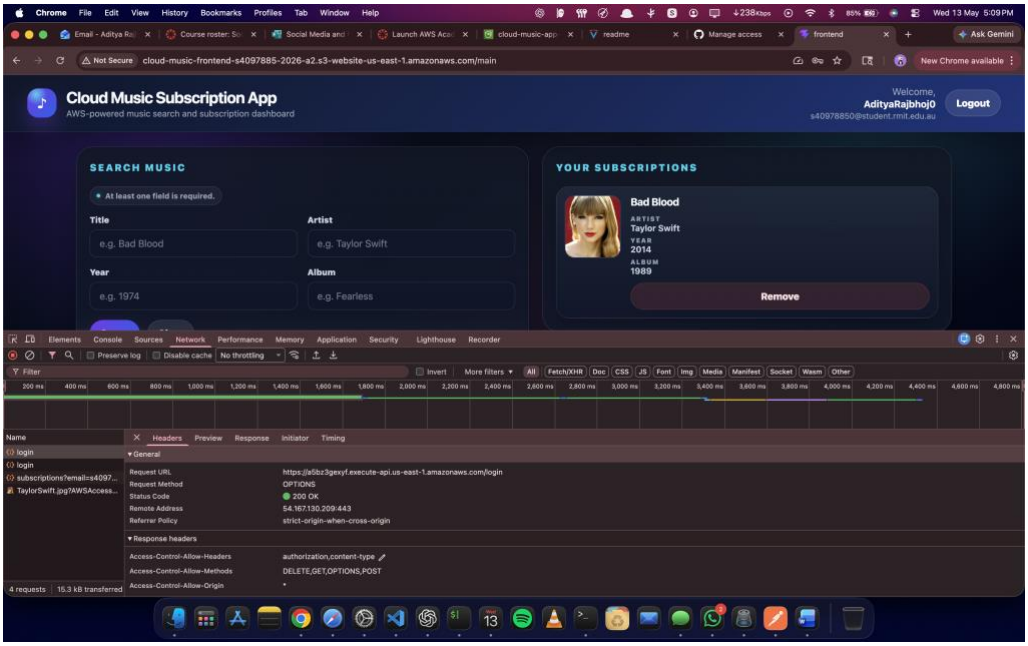

Figure 10. Frontend validation showing the S3-hosted React application successfully connected to the API Gateway backend for login, search, and subscription operations.

Figure 10 confirms that the deployed frontend was not only loading as a static page but was also connected to a working AWS backend. This validates the end-to-end flow between S3 static hosting, API Gateway, Lambda, DynamoDB, and S3 artist image storage.

5. Backend Deployment Architectures

Our backend was deployed using three separate AWS architectures to demonstrate different cloud deployment models for the same application logic. We implemented a traditional server-based deployment using Amazon EC2, a containerised deployment using Amazon ECS/Fargate, and a serverless deployment using API Gateway with AWS Lambda. All three backends connect to the same DynamoDB tables and S3 artist image bucket, which allowed us to compare deployment models without changing the core application behaviour.

5.1. EC2 Deployment

The first backend deployment used Amazon EC2 to run the Flask API as a traditional cloud-hosted server. This architecture gave us direct control over the runtime environment, including package installation, environment configuration, server process management, and network access. The EC2 instance was launched in the `us-east-1` region using an Amazon Linux environment, with an IAM instance profile that allowed the backend to access DynamoDB and S3.

After connecting to the instance through SSH, we cloned the project repository, installed the required Python dependencies, created the production environment configuration, and verified that the backend could access the shared DynamoDB tables. The backend was then started using Gunicorn on standard HTTP port 80. This allowed the API to be accessed through the EC2 public IP address without requiring a custom port in the frontend configuration.

The EC2 backend used the same API routes as the local backend, including `/health`, `/login`, `/register`, `/songs`, `/subscriptions`, and `/subscriptions/<email>/<song_id>`. This made it possible to test the EC2 deployment using the same functional workflow as the ECS and Lambda deployments. The `/health` endpoint was used as the main deployment check because it confirmed that the EC2 security group, public networking, Gunicorn process, and Flask application were functioning correctly.

The EC2 deployment demonstrated that the backend could run successfully as a persistent server-based application while using the same DynamoDB and S3 resources as the other architectures. Its main advantage was operational control because the server could be inspected and managed directly through SSH. However, this model also required more manual responsibility than the ECS and Lambda deployments, including instance management, patching, process control, and shutdown to avoid unnecessary AWS Academy credit usage.

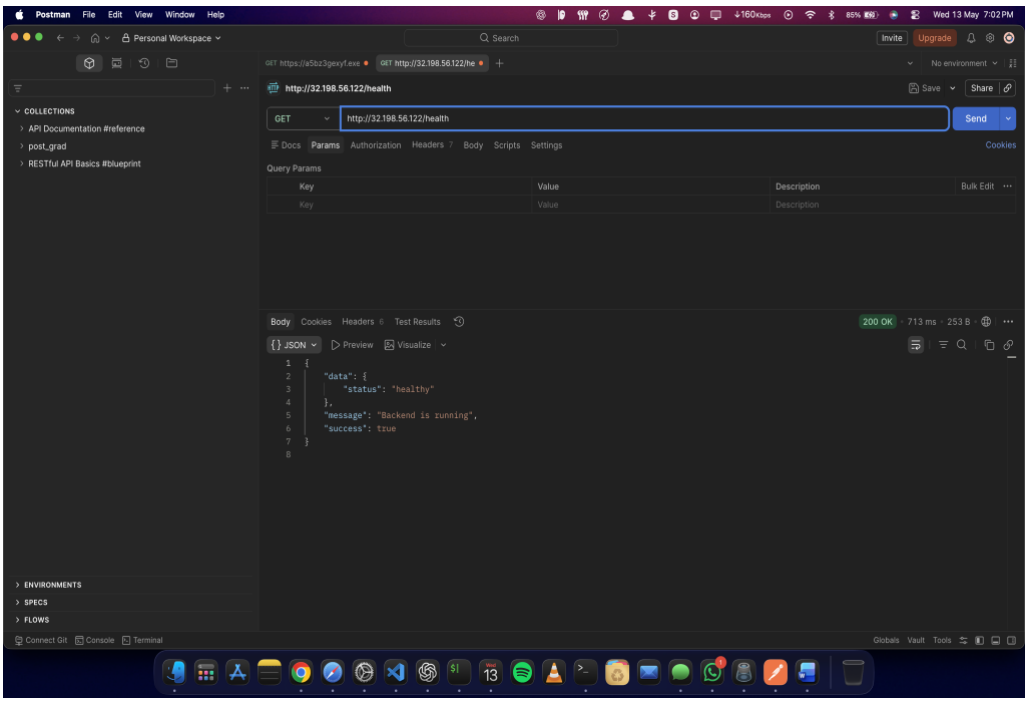

Figure 11. EC2 backend health-check output confirming that the Flask API deployed on Amazon EC2 is reachable over HTTP and returning a successful response.

Figure 11 confirms that the EC2 backend was reachable through its public IP address and returned a valid response from the `/health` endpoint. This verifies that the EC2 instance, security group, Gunicorn process, and Flask application were configured correctly.

5.2.ECS/Fargate Deployment

The second backend deployment used Amazon ECS with AWS Fargate to run the same Flask API as a containerised service. In this architecture, the backend application was packaged into a Docker image, pushed to Amazon Elastic Container Registry, and then deployed as an ECS Fargate task. This approach improved deployment consistency because the application runtime, dependencies, and startup command were defined inside the container image rather than configured manually on a server.

Our ECS deployment script automated the main deployment steps. It checked AWS credentials and Docker availability, created or reused the ECR repository, built the backend image for the Linux AMD64 platform, pushed the image to ECR, created the ECS cluster, configured the Fargate task definition, and started the ECS service. The task definition passed the required environment variables for DynamoDB table names, index names, S3 image bucket configuration, CORS settings, and application port configuration. This allowed the ECS backend to connect to the same `login`, `music`, `subscriptions`, and S3 artist image resources used by the other deployments.

The ECS service was deployed with a public network configuration and an HTTP security group rule on port 80. After deployment, the script retrieved the running task's public IP address and tested the `/health` endpoint. This confirmed that the container started successfully, Gunicorn was listening on port 80, the ECS networking configuration was valid, and the backend could respond to external API requests.

Compared with EC2, ECS/Fargate reduced the need to manage the operating system directly. We did not need to SSH into a server to configure the runtime once the image and task definition were prepared. This made the deployment more reproducible and closer to a professional cloud-native workflow. However, it also introduced additional components, including Docker, ECR, task definitions, ECS services, and CloudWatch logs, which made the deployment more complex than the EC2 version.

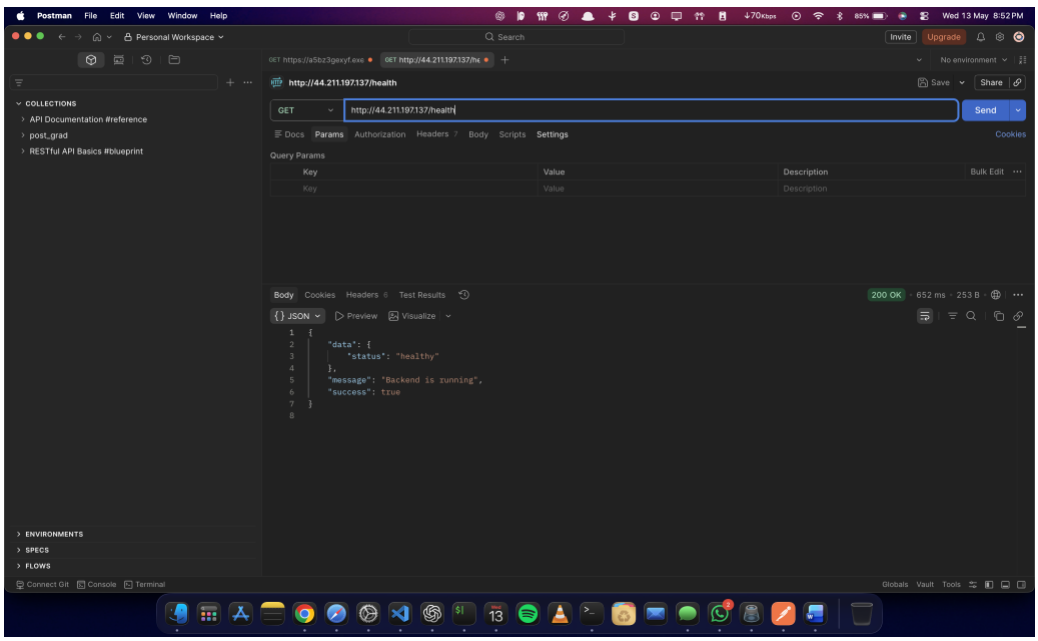

Figure 12. ECS/Fargate backend health-check output confirming that the containerised Flask API is running successfully as a Fargate task and responding over HTTP.

Figure 12 confirms that the ECS/Fargate backend was deployed successfully and could be reached through the public task IP address. The successful `/health` response verifies that the Docker image, ECR push, Fargate task, security group, and backend application were configured correctly.

5.3.API Gateway and Lambda Deployment

The third backend deployment used API Gateway with AWS Lambda to implement the serverless version of the application backend. In this architecture, API Gateway exposes the HTTP endpoints and forwards requests to a Lambda function, while Lambda executes the backend routing logic and interacts with DynamoDB and S3. This removed the need to manage a virtual server or a running container for the backend.

For this deployment, we implemented a native Lambda router in `deployments/lambda/lambda_function.py`. This was used instead of running the full Flask server inside Lambda, which made the serverless implementation more direct and reduced unnecessary runtime complexity. The Lambda package included the required backend services for authentication, music search, subscriptions, DynamoDB access, and S3 image URL generation. The deployment script packaged the Lambda function, configured the required environment variables, assigned the LabRole, and validated the function with a direct `/health` invocation.

API Gateway was then configured as the public HTTPS entry point for the Lambda backend. The API routes included `/health`, `/login`, `/register`, `/songs`, `/subscriptions`, and `/subscriptions/{email}/{song_id}`. These routes mapped the required GET, POST, and DELETE operations to the Lambda function. This allowed the frontend to call a single API Gateway base URL while the Lambda function handled routing internally.

The serverless backend was validated using both direct Lambda invocation and API Gateway requests. The `/health` endpoint confirmed that API Gateway could invoke Lambda successfully, while login, music search, subscription creation, and subscription deletion tests confirmed that Lambda could access DynamoDB and S3 correctly. This deployment was selected as the final demo backend because it showed a fully managed cloud path from the S3-hosted frontend to API Gateway, Lambda, DynamoDB, and S3 image storage.

Compared with EC2 and ECS, the API Gateway and Lambda deployment required the least server management. There was no persistent backend server to maintain, patch, or manually restart. However, the serverless model required careful request routing, environment variable configuration, permission setup, and response formatting. In our implementation, this architecture provided the cleanest final demo flow and the most cost-conscious runtime model because the backend only executes when requests are made.

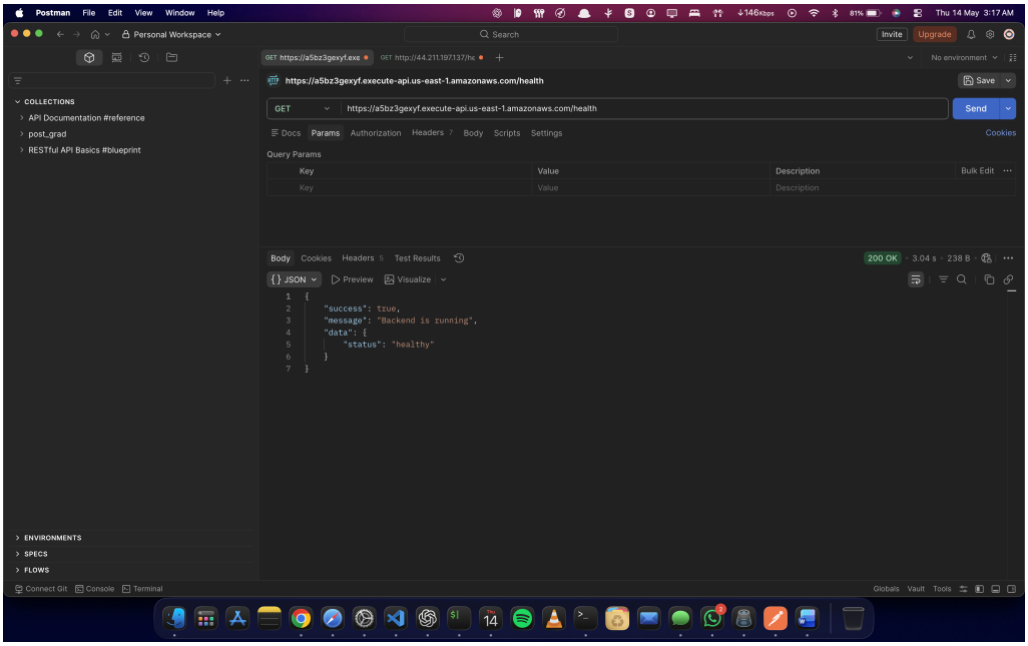

Figure 13. API Gateway and Lambda backend validation showing the serverless backend responding successfully through the deployed HTTPS API endpoint.

Figure 13 confirms that API Gateway successfully routed the request to the Lambda backend and returned a valid JSON response. This verifies that the serverless route mapping, Lambda integration, IAM permissions, and backend routing logic were configured correctly.

6. Functional Testing and Validation

Our application was validated through a combination of backend API testing, frontend interaction testing, and AWS service verification. Rather than testing each AWS component in isolation only, we focused on confirming that the complete application workflow operated correctly across the deployed system. The key workflows tested were login, registration, music search, subscription creation, subscription retrieval, subscription deletion, and artist image display.

The backend deployments were first tested using their `/health` endpoints to confirm that each architecture was reachable. The EC2 backend, ECS/Fargate backend, and API Gateway with Lambda backend each returned successful responses, confirming that all three deployment models were operational. The API Gateway and Lambda backend was then used for final end-to-end validation because it represented the selected demo configuration and provided a fully managed path from the S3-hosted frontend to Lambda, DynamoDB, and S3.

Functional testing confirmed that the login route could retrieve valid user records from the `login` table, while the registration route could create a new user when the email was unique. Music search was tested using query combinations such as artist and album, and artist and year. These tests confirmed that the backend could retrieve records from the `music` table using the implemented key and index strategy. Subscription testing confirmed that the application could create, retrieve, and delete records in the `subscriptions` table using the logged-in user's email and the selected `song_id`.

The frontend was also tested through the deployed S3 website. After logging in, we searched for songs, viewed returned artist images, subscribed to a song, removed a subscription, and logged out. This confirmed that the frontend was not only loading from S3, but was also correctly communicating with the configured backend endpoint and updating the interface based on live API responses.

Table 5. Functional validation summary

Test area	Validation performed	Evidence used in report	Result
Frontend interface	S3-hosted React dashboard loaded and displayed search/subscription areas	Figure 1	Passed
API Gateway health	`/health` returned HTTP 200 and backend status message	Figure 4	Passed
DynamoDB setup	Login, music, subscription tables and indexed queries verified	Figure 6	Passed
S3 image storage	Artist images uploaded under `artist-images/` prefix	Figure 7	Passed
EC2 backend	EC2 `/health` endpoint returned successful response	Figure 11	Passed
ECS backend	ECS/Fargate `/health` endpoint returned successful response	Figure 12	Passed
Lambda backend	API Gateway request successfully invoked Lambda backend	Figure 13	Passed
Music search	Search returned expected records such as Taylor Swift in *Fearless*	API/Postman validation	Passed
Subscription workflow	Add, retrieve, and remove subscription operations completed successfully	API/Postman validation	Passed

Overall, the testing confirmed that the application was deployed and operational across all required AWS backend models. The results also confirmed that the shared DynamoDB and S3 data layer supported the main user workflows consistently, regardless of whether the backend was running on EC2, ECS/Fargate, or API Gateway with Lambda.

7. Critical Comparison of Backend Architectures

Our project deployed the same backend functionality using three AWS architectures: Amazon EC2, Amazon ECS/Fargate, and API Gateway with AWS Lambda. This allowed us to compare the deployment models fairly because each backend supported the same core operations: login, registration, music search, subscription creation, subscription retrieval, subscription deletion, and S3 artist image URL generation. All three implementations connected to the same DynamoDB tables and S3 artist image bucket, so the comparison focuses on infrastructure design rather than differences in application behaviour.

The EC2 deployment gave us the highest level of direct control. We could SSH into the Amazon Linux instance, install dependencies, configure the environment, verify DynamoDB access, and run the Flask backend through Gunicorn on port 80. This made EC2 useful for troubleshooting because server state, public IP address, security group rules, and the backend process could be inspected manually. However, this model also created the most operational responsibility. The instance had to be started, stopped, secured, monitored, and managed directly. Since the public IP can change after restart and the backend process must remain running, EC2 was useful as a baseline deployment but less convenient as the final demonstration backend.

The ECS/Fargate deployment improved repeatability by packaging the backend into a Docker image and storing it in Amazon ECR. This was a stronger deployment model than EC2 because the runtime, dependencies, and Gunicorn startup command were captured inside the container image instead of being configured manually on a virtual machine. The ECS deployment script created or reused the ECR repository, built and pushed the image, registered the task definition, configured the service, and validated the public health endpoint. This made ECS/Fargate more professional and reproducible, but it also introduced more components to manage, including Docker, ECR, task definitions, service stability, security groups, network interfaces, public task IP retrieval, and CloudWatch logs.

The API Gateway and Lambda deployment was the best fit for our final application because the backend workload is lightweight and request-driven. The system does not require persistent server-side state or long-running computation. Most operations are short API requests that read from DynamoDB, write or delete a subscription record, or return an S3 image URL. Lambda matched this pattern well because compute is triggered only when requests are made. API Gateway also provided a stable HTTPS endpoint for the S3-hosted React frontend, which made the final deployed system easier to demonstrate than relying on changing EC2 or ECS public IP addresses.

Cost and operational overhead were also important in our AWS Academy environment. EC2 can consume credit while the instance is running, even if the application is idle. ECS/Fargate also consumes resources while the service task is running. API Gateway with Lambda is more cost-conscious for intermittent testing and demonstration because there is no continuously running backend server or container. This made the serverless architecture the most practical final deployment model for our project, while EC2 and ECS could be stopped or scaled down after validation.

Each architecture had a clear role in the final system. EC2 demonstrated a traditional server-based deployment with maximum control. ECS/Fargate demonstrated a containerised deployment with stronger reproducibility and more professional release packaging. API Gateway with Lambda provided the cleanest final architecture for this specific application because it reduced infrastructure management, offered a stable HTTPS API, and aligned well with short stateless backend requests.

Table 6. Comparison of backend deployment architectures

Criteria	EC2 backend	ECS/Fargate backend	API Gateway + Lambda backend
Deployment model	Flask/Gunicorn on an Amazon Linux virtual server	Dockerised Flask/Gunicorn backend running as a Fargate task	Native Lambda router exposed through API Gateway
Fit for our workload	Works, but requires a continuously running instance	Works, but requires a running container service	Best fit for short, stateless, request-driven API calls
Runtime control	Highest control through SSH and direct server inspection	Controlled through Docker image, task definition, and ECS service	Lowest direct runtime control, but least infrastructure management
Deployment repeatability	Moderate; depends on server setup and instance state	High; Docker image captures runtime and dependencies	High; packaged Lambda deployment and scripted API Gateway setup
Operational effort	Highest; requires instance, process, security, and restart management	Moderate; requires Docker, ECR, ECS service, networking, and logs	Lowest after setup; no server or container needs to be maintained
Public access	Public EC2 IP on port 80	Public Fargate task IP on port 80	Stable HTTPS API Gateway endpoint
Scaling approach	Manual unless Auto Scaling and Load Balancer are added	Scales through ECS service/task configuration	Request-based scaling managed by AWS
Cost behaviour in this project	Cost risk if the instance is left running	Cost risk if the Fargate task remains running	Most suitable for intermittent demo/testing usage

Criteria	EC2 backend	ECS/Fargate backend	API Gateway + Lambda backend
Troubles hooting method	SSH, process checks, security group checks, curl/Postman tests	ECS events, task status, ENI/public IP checks, CloudWatch logs	API Gateway responses, Lambda direct invoke, CloudWatch logs
Main limitation	Manual server management and changing public IP after restart	More deployment components and setup complexity	Requires correct event routing, permissions, and response formatting
Best role in our project	Baseline server deployment	Professional containerised deployment	Preferred final demo backend

Final architecture decision:
For the final demo configuration, we selected **API Gateway with Lambda** as the preferred backend. This decision was justified by the nature of our application: the backend performs short stateless API operations, DynamoDB handles persistence, S3 stores artist images, and the system does not need a continuously running application server. EC2 and ECS/Fargate remained important because they demonstrated that the same backend could be deployed across traditional, containerised, and serverless AWS architectures. However, for our final cloud application, the serverless model offered the strongest balance of availability, cost-awareness, maintainability, and demonstration clarity.

8. Security, Cost, Scalability, and Operational Considerations

Our implementation was designed as a functional cloud application suitable for an academic AWS environment. The main priority was to demonstrate correct integration between the frontend, backend, DynamoDB, S3, and the three backend deployment models. At the same time, we considered security, cost, scalability, and operational behaviour so that the system could be evaluated beyond basic functionality.

8.1. Security Considerations

The application uses AWS IAM roles and environment variables to separate configuration from application logic. The backend deployments access DynamoDB and S3 through AWS Academy credentials, environment configuration, or assigned IAM roles depending on the deployment model rather than hardcoding access keys into the source code. Sensitive local configuration, such as `.env` files and AWS credentials, was excluded from the repository to reduce the risk of exposing secrets.

For the frontend, the final React build is hosted in an S3 static website bucket. Since static website assets must be publicly readable for users to access the site, the frontend bucket is

configured for public website access. This is acceptable for static compiled frontend files, but it is different from the artist image bucket. Artist images are stored separately in the application image bucket, and the backend returns generated S3 image URLs when music records are retrieved.

The deployed backends also use CORS configuration so the S3-hosted frontend can communicate with the API endpoints. For the assignment prototype, permissive CORS was used to simplify testing across EC2, ECS, and API Gateway endpoints. In a production environment, CORS should be restricted to the final frontend domain only. Similarly, the current authentication approach is suitable for demonstrating application functionality, but production systems should store hashed passwords and use stronger session management such as JWT or managed authentication.

8.2.Cost Considerations

Cost control was important because the project was developed in an AWS Academy environment with limited credits. Our design avoided unnecessary large compute resources and used lightweight services where possible. DynamoDB was suitable for the small dataset because the application only stores 137 music records, default users, and user subscription records. S3 was also appropriate because the frontend build files and artist images are small static objects.

The three backend architectures have different cost behaviours. EC2 can continue consuming credit while the instance is running, even when no user is accessing the application. ECS/Fargate can also consume resources while the service task remains active. For this reason, both EC2 and ECS were stopped or scaled down after testing. API Gateway with Lambda was more suitable for final demonstration because it does not require a continuously running backend server or container, making it more cost-conscious for intermittent testing and demo usage.

8.3.Scalability Considerations

The frontend layer is naturally scalable because it is hosted as static content in Amazon S3. Users retrieve static HTML, CSS, JavaScript, and asset files directly from the S3 website endpoint. The backend layer differs depending on the deployment model. EC2 provides limited scalability unless additional infrastructure such as a load balancer and auto scaling group is added. ECS/Fargate is more scalable because additional tasks can be configured as demand increases. API Gateway with Lambda provides the simplest scaling model for this application because requests can be handled without managing servers or container capacity directly.

DynamoDB also supports scalable data access patterns when table keys and indexes are designed around expected queries. Our use of an artist-based partition key, generated `song_id`, title-based GSI, and artist-year/album LSI supports the main application search and subscription workflows. While the dataset is small, the schema design avoids obvious key risks such as using title alone as a unique identifier.

8.4.Operational Considerations

Operationally, the project benefited from deployment scripts for repeatability. The frontend S3 deployment, ECS deployment, Lambda packaging, and API Gateway setup were scripted so the system could be redeployed with fewer manual steps. This was especially useful during testing because backend endpoints changed depending on the selected deployment model.

Each backend architecture required a different operational approach. EC2 required manual instance management and process control. ECS required Docker image management, ECR repository usage, ECS task definitions, and service monitoring. Lambda required correct packaging, IAM permissions, API Gateway route configuration, and CloudWatch log inspection. These differences helped us understand the trade-offs between control, repeatability, and operational simplicity.

Table 7. Security, cost, scalability, and operational summary

Area	Implementation decision	Evaluation
Frontend hosting	S3 static website hosting	Simple, low-cost, and suitable for React static assets
Backend security	AWS roles/environment configuration	Avoids hardcoded credentials in source code
CORS	Permissive for assignment testing	Works for demo, but should be restricted in production
Authentication	Basic email/password application login	Suitable for prototype; production should use password hashing and stronger sessions
Cost control	EC2 stopped and ECS scaled down after testing	Reduces unnecessary AWS Academy credit usage
Scalability	Lambda/API Gateway selected for final demo backend	Best fit for short, stateless, request-driven API operations
Operations	Deployment scripts used for repeatability	Reduces manual deployment errors and improves demo readiness

Overall, the implementation is appropriate for the scope of the assignment and demonstrates a practical balance between functionality, cost awareness, and cloud architecture quality. The main production improvements would be stronger authentication, stricter CORS rules, custom domain and HTTPS configuration for the frontend, and more detailed monitoring across all backend deployments.

9. Limitations and Future Improvements

Although the Cloud Music Subscription App meets the functional and deployment requirements of the assignment, it remains an academic prototype and would require further hardening before production use. The main limitations relate to authentication, security configuration, endpoint stability, monitoring, and scalability beyond the provided dataset.

The current authentication model supports registration and login, but it is intentionally simple for demonstrating application functionality. In a production system, passwords should be securely hashed, and sessions should be managed through signed tokens or a managed identity service. Similarly, CORS was configured permissively to support testing across EC2, ECS/Fargate, and API Gateway endpoints. A production deployment should restrict allowed origins to the final frontend domain.

Public endpoint management is another limitation. The EC2 and ECS deployments used public IP addresses for testing, which can change when resources restart. A more robust design would use HTTPS, a custom domain, and either a load balancer or managed API endpoint. Monitoring could also be improved by adding structured logs, alerts, and dashboard metrics for API errors, latency, failed logins, DynamoDB performance, and backend availability.

Finally, the dataset is small and static, which is appropriate for the assignment but may not reflect production-scale usage. If the application were expanded, the query strategy should be reviewed for larger datasets, higher user activity, pagination, caching, additional indexes, and stronger input validation.

Future improvements would therefore include password hashing, token-based authentication, stricter CORS rules, HTTPS with a custom domain, stronger monitoring, automated CI/CD deployment, improved frontend error handling, and further optimisation of DynamoDB access patterns.

10. Conclusion

The Cloud Music Subscription App successfully demonstrates a deployed AWS-based web application integrating React frontend hosting, backend APIs, DynamoDB data storage, S3 object storage, and three backend deployment models. The application supports registration, login, music search, subscription management, and artist image display through the S3-hosted frontend.

A key strength of the implementation was the data-layer design. By analysing the `2026a2_songs.json` dataset before import, we identified duplicate title risks and avoided unsafe key choices. The final DynamoDB design used separate `login`, `music`, and `subscriptions` tables, a generated `song_id`, and purposeful indexing to support accurate search and subscription operations.

The backend was deployed through EC2, ECS/Fargate, and API Gateway with Lambda. Each deployment was validated independently while using the same DynamoDB and S3 resources.

EC2 provided direct server control, ECS/Fargate provided containerised repeatability, and API Gateway with Lambda offered the most suitable final architecture for the application's short, stateless API workload.

Overall, the project met its technical objectives and demonstrated the trade-offs between server-based, containerised, and serverless AWS deployment models. The final system is functional, cost-conscious for an AWS Academy environment, and supported by evidence across the frontend, backend, database, and storage layers.

11. References

Amazon Web Services. (n.d.). *Amazon API Gateway developer guide*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html

Amazon Web Services. (n.d.). *Amazon EC2 instances*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Instances.html

Amazon Web Services. (n.d.). *Amazon Elastic Container Registry user guide: What is Amazon Elastic Container Registry?* AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html

Amazon Web Services. (n.d.). *Architect for AWS Fargate for Amazon ECS*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

Amazon Web Services. (n.d.). *AWS Fargate or AWS Lambda?* AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/decision-guides/latest/fargate-or-lambda/fargate-or-lambda.html

Amazon Web Services. (n.d.). *Hosting a static website using Amazon S3*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html

Amazon Web Services. (n.d.). *Improving data access with secondary indexes in DynamoDB*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html

Amazon Web Services. (n.d.). *Local secondary indexes*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/LSI.html

Amazon Web Services. (n.d.). *Scan*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Scan.html

Amazon Web Services. (n.d.). *Using global secondary indexes in DynamoDB*. AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html

Amazon Web Services. (n.d.). *What is Amazon EC2?* AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html

Amazon Web Services. (n.d.). *What is Amazon Elastic Container Service?* AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

Amazon Web Services. (n.d.). *What is AWS Lambda?* AWS Documentation. Retrieved May 14, 2026, from https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

Docker Inc. (n.d.). *Dockerfile reference*. Docker Docs. Retrieved May 14, 2026, from https://docs.docker.com/reference/dockerfile/

Meta Open Source. (n.d.). *Quick start*. React Documentation. Retrieved May 14, 2026, from https://react.dev/learn

Vite. (n.d.). *Getting started*. Vite Documentation. Retrieved May 14, 2026, from https://vite.dev/guide/